

21. Spring Cloud

21.1. Giới thiệu về Spring Cloud

Spring Cloud là một tập hợp các công cụ và framework được xây dựng trên Spring Boot, nhằm mục đích đơn giản hóa việc phát triển các ứng dụng phân tán, đặc biệt là trong kiến trúc microservices. Nó cung cấp các giải pháp cho các vấn đề chung mà các hệ thống phân tán phải đối mặt, như:

- **Service Discovery:** Làm thế nào để các dịch vụ tìm thấy nhau?
- **Configuration Management:** Làm thế nào để quản lý cấu hình tập trung cho nhiều dịch vụ?
- **Load Balancing:** Làm thế nào để phân phối request giữa các instance của dịch vụ?
- **Circuit Breakers:** Làm thế nào để xử lý lỗi và ngăn chặn sự cố lan truyền?
- **API Gateway:** Làm thế nào để cung cấp một điểm truy cập duy nhất cho các client?
- **Distributed Tracing:** Làm thế nào để theo dõi request qua nhiều dịch vụ?
- **Message Bus:** Làm thế nào để giao tiếp bất đồng bộ giữa các dịch vụ?

Spring Cloud không phải là một giải pháp "tất cả trong một" mà là một tập hợp các dự án con, mỗi dự án giải quyết một khía cạnh cụ thể của kiến trúc microservices. Nó tận dụng các dự án mã nguồn mở phổ biến như Netflix OSS, HashiCorp Consul, Apache Zookeeper, v.v., và tích hợp chúng một cách liền mạch với Spring Framework.

21.2. Các dự án con chính của Spring Cloud

21.2.1. Spring Cloud Netflix

Spring Cloud Netflix cung cấp tích hợp cho các thành phần Netflix OSS phổ biến:

- **Eureka (Service Discovery):** Cho phép các dịch vụ đăng ký và khám phá lẫn nhau. Eureka Server là trung tâm đăng ký, và Eureka Client là các dịch vụ đăng ký với server.

- **Ribbon (Client-side Load Balancer):** Cung cấp khả năng cân bằng tải phía client, tích hợp với Eureka để tìm các instance dịch vụ có sẵn.
- **Feign (Declarative REST Client):** Giúp tạo các client RESTful một cách khai báo, đơn giản hóa việc gọi các dịch vụ khác.
- **Hystrix (Circuit Breaker):** Cung cấp khả năng chịu lỗi và độ trễ bằng cách cô lập các điểm truy cập vào các dịch vụ từ xa, hệ thống bên thứ ba và ngăn chặn lỗi lan truyền (hiện đã ở chế độ bảo trì, thay thế bằng Resilience4j).
- **Zuul (API Gateway):** Cung cấp các tính năng định tuyến, lọc và bảo mật cho các request đến các microservices (hiện đã ở chế độ bảo trì, thay thế bằng Spring Cloud Gateway).

21.2.2. Spring Cloud Config

Spring Cloud Config Server: Cung cấp một máy chủ cấu hình tập trung, phân tán, hỗ trợ các backend như Git, Subversion, hoặc hệ thống file cục bộ. Nó cho phép các microservices lấy cấu hình của chúng từ một vị trí tập trung, giúp quản lý cấu hình dễ dàng hơn trong môi trường phân tán.

Spring Cloud Config Client: Các microservices sử dụng Config Client để kết nối với Config Server và tải cấu hình của chúng.

21.2.3. Spring Cloud Gateway

Spring Cloud Gateway: Một API Gateway hiện đại, hiệu suất cao, được xây dựng trên Spring Framework 5, Spring Boot 2 và Project Reactor. Nó cung cấp các tính năng định tuyến, lọc, rewrite URL, giới hạn tốc độ, và bảo mật.

21.2.4. Spring Cloud Bus

Spring Cloud Bus: Liên kết các node của hệ thống phân tán với một message broker (ví dụ: RabbitMQ, Kafka). Điều này cho phép các sự kiện cấu hình hoặc các sự kiện quản lý khác được truyền bá giữa các dịch vụ, ví dụ: làm mới cấu hình của tất cả các dịch vụ khi cấu hình trên Config Server thay đổi.

21.2.5. Spring Cloud Stream

Spring Cloud Stream: Một framework để xây dựng các ứng dụng xử lý sự kiện dựa trên message. Nó cung cấp một cách trừu tượng hóa để kết nối với các message broker như Kafka hoặc RabbitMQ, giúp đơn giản hóa việc phát triển các microservices giao tiếp thông qua message.

21.2.6. Spring Cloud Sleuth

Spring Cloud Sleuth: Cung cấp khả năng Distributed Tracing cho các ứng dụng Spring. Nó tự động thêm các ID truy vết (trace ID) và ID span (span ID) vào các log và header HTTP, cho phép bạn theo dõi luồng của một request qua nhiều microservices. Sleuth tích hợp với các hệ thống tracing như Zipkin hoặc OpenTelemetry.

21.2.7. Spring Cloud OpenFeign

Spring Cloud OpenFeign: Một client RESTful khai báo, giúp đơn giản hóa việc gọi các dịch vụ REST khác. Nó tích hợp với Ribbon (hoặc Spring Cloud LoadBalancer) để cân bằng tải phía client và Hystrix (hoặc Resilience4j) để xử lý lỗi.

21.2.8. Spring Cloud CircuitBreaker (Resilience4j)

Spring Cloud CircuitBreaker: Cung cấp một cách trừu tượng hóa để sử dụng các thư viện Circuit Breaker như Resilience4j. Resilience4j là một thư viện nhẹ, dễ sử dụng, cung cấp các tính năng Circuit Breaker, Rate Limiter, Retry, Bulkhead, và TimeLimiter.

21.3. Ví dụ tích hợp Spring Cloud Config

Để minh họa, chúng ta sẽ xem xét một ví dụ đơn giản về việc sử dụng Spring Cloud Config để quản lý cấu hình tập trung.

21.3.1. Config Server

pom.xml:

XML

```
<dependency>  
  <groupId>org.springframework.cloud</groupId>
```

```
<artifactId>spring-cloud-config-server</artifactId>
</dependency>
```

Application Class:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;

@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }
}
```

application.yml:

YAML

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/spring-cloud/spring-cloud-config-samples #
          Thay bằng repo của bạn
          search-paths: config-repo # Thư mục chứa cấu hình trong repo
```

21.3.2. Config Client (Microservice)

pom.xml:

XML

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-config</artifactId>
```

```
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

bootstrap.yml (quan trọng!):

File `bootstrap.yml` (hoặc `bootstrap.properties`) được tải trước `application.yml` và được sử dụng để cấu hình các thuộc tính liên quan đến Spring Cloud.

YAML

```
spring:
  application:
    name: my-client-service # Tên ứng dụng, dùng để tìm cấu hình trên Config
Server
  cloud:
    config:
      uri: http://localhost:8888 # Địa chỉ của Config Server
      profile: dev # Profile để lấy cấu hình (ví dụ: my-client-service-
dev.yml)
```

Application Class:

Java

```
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ConfigClientApplication {
    public static void main(String[] args) {
        SpringApplication.run(ConfigClientApplication.class, args);
    }
}
```

Controller để đọc cấu hình:

Java

```
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class MessageController {

    @Value("${message:Default Message}")
    private String message;

    @GetMapping("/message")
    public String getMessage() {
        return message;
    }
}
```

Khi `my-client-service` khởi động, nó sẽ kết nối với Config Server tại `http://localhost:8888` và tìm file cấu hình `my-client-service-dev.yml` (hoặc `.properties`) trong kho Git. Giá trị của thuộc tính `message` sẽ được inject vào `MessageController`.

21.4. Bản chất của Spring Cloud

Bản chất của Spring Cloud là cung cấp một bộ công cụ và framework toàn diện để giải quyết các thách thức cố hữu của việc xây dựng và vận hành các ứng dụng phân tán, đặc biệt là trong kiến trúc microservices. Nó hoạt động như một lớp trừu tượng trên các dự án mã nguồn mở khác nhau (như Netflix OSS, HashiCorp Consul, RabbitMQ, Kafka, Zipkin), tích hợp chúng một cách liền mạch với Spring Boot. Điều này giúp các nhà phát triển dễ dàng áp dụng các mẫu thiết kế phân tán (distributed patterns) mà không cần phải lo lắng về các chi tiết triển khai phức tạp. Spring Cloud cho phép xây dựng các hệ thống microservices có khả năng mở rộng, chịu lỗi, dễ quản lý và giám sát, từ đó tăng tốc độ phát triển và triển khai các ứng dụng hiện đại.